Project Report

On

Python Programming

Subject Code: 26CAH-606

PROJECT TOPIC

RETAIL SALES & CUSTOMER BEHAVIOR EXPLORATORY DATA ANALYSIS (EDA) SYSTEM

Submitted By:

Name: Muskan Gupta

UID: 26MCD10014

Section: 26MCD1

Group: A

Submitted To:

Dr.Pawandeep Sharma

Assistant Professor

E2916

University Institute of Computing

Chandigarh University, Gharuan, Mohali

UNIVERSITY INSTITUTE OF COMPUTING

Discover. Learn. Empower.

NAAC GRADE A+

Accredited University

BONAFIDE CERTIFICATE

Certified that this project report entitled **"Retail Sales & Customer Behavior Exploratory Data Analysis (EDA) "** is the bonafide work of "**Muskan Gupta, UID 26MCD10014"**, who has carried out the project work under the supervision of the concerned faculty member. The content and findings presented herein reflect the work undertaken as part of the academic requirements of the course.

The project demonstrates the practical application of Python programming, Object-Oriented Programming (OOP) with inheritance, NumPy, Pandas, and Matplotlib for retail sales exploratory data analysis, statistical calculations, customer behavior insights, and 2x2 dashboard visualization.

Project Supervisor:

Dr.Pawandeep Sharma

(E2916)

Head of Department:

Dr.Amanpreet Kaur Sandhu

HOD - MCA

UNIVERSITY INSTITUTE OF COMPUTING
Discover. Learn. Empower.

NAAC GRADE A+
Accredited University

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all the individuals who have contributed to the successful completion of this project. Developing this project provided me with an opportunity to improve my knowledge of Python programming and data analysis.

I would like to extend my sincere appreciation to **Dr. Amanpreet Kaur Sandhu,** Head of Department MCA and faculty members for their valuable support and guidance throughout the duration of this project. Their encouragement and suggestions helped me complete my work successfully.

I owe special thanks to my project supervisor **Dr. Pawandeep Sharma, Academic Coordinator, University Institute of Computing,** for providing guidance, support, and motivation during the development of this project. Their valuable suggestions helped me understand the concepts and improve my project.

I would also like to thank all my teachers and friends for their cooperation and encouragement. Finally, I am grateful to my parents for their continuous support and motivation throughout my academic journey.

Muskan Gupta

UNIVERSITY INSTITUTE OF COMPUTING
Discover. Learn. Empower.

NAAC GRADE A+
Accredited University

Index

S.No.	Contents	Page No.
1.	Bonafide Certificate	2
2.	Acknowledgement	3
3.	Introduction	5
4.	Problem Statement	5
5.	Objectives of the Project	5
6.	Scope of the Project	6
7.	Hardware and Software Requirements	6
8.	Technologies and Libraries Used	7
9.	System Architecture and Objects Designed	7
10.	Functional Work Algorithm	7
11.	Python Source Code	8
12.	Output	12
13.	Graphical Dashboard Analysis	12
14.	System Advantages	13
15.	Project Limitations	13
16.	Key Learning Outcomes	14
17.	Conclusion	14

1. Introduction

In modern retail management, analyzing monthly sales figures, category-wise product performances, and buyer demographics is critical for strategic decision-making. Traditional manual bookkeeping and basic spreadsheet approaches suffer from calculation errors, lack scalability, and fail to provide visual insights required for rapid executive evaluation.

The **Retail Sales & Customer Behavior Exploratory Data Analysis (EDA) System** resolves these challenges by introducing a modular, computerized Python application. Built specifically using single-inheritance Object-Oriented Design, the application ingests transactional datasets, cleans missing records, computes statistical dispersion metrics via NumPy, and renders a comprehensive 2×2 visual analytics dashboard using Matplotlib.

2. Problem Statement

Retail organizations collect vast amounts of sales data; however, converting raw numerical lists into actionable business insights remains difficult without complex enterprise software. Small to medium businesses require a lightweight, self-contained Python application that cleans raw input, automates total/mean statistics, and visualizes sales distribution across product lines and age brackets in a single execution.

3. Objectives of the Project

The primary technical and analytical objectives of this project include:

- **OOP Implementation:** To construct a robust modular architecture utilizing a Parent Class (DatasetManager) and a Child Class (RetailSalesAnalyzer) demonstrating inheritance and super() calls.
- **Automated Data Cleaning:** To validate missing values and implement non-destructive re-assignment cleaning using df = df.dropna().
- **Numerical Computing:** To compute exact statistical measures (Total, Mean, Median, and Standard Deviation) using NumPy routines.
- **Multi-Panel Dashboarding:** To generate a clear 2×2 Matplotlib dashboard containing line plots, bar charts, pie charts, and scatter plots.
- **Syllabus Alignment:** To meet MCA Data Science requirements by combining core Python structures with scientific libraries without unnecessary external GUI dependencies.

4. Scope of the Project

The system handles complete end-to-end exploratory data processing for retail records. It accepts structured transactional dictionaries consisting of Customer ID, Month, Category, Revenue, Units Sold, and Customer Age.

Current Capabilities:

- 12-month complete longitudinal dataset processing.
- Automated data cleaning and missing value checks.
- NumPy-powered statistical summary generation.
- Groupby aggregations for category revenue totals.
- Automated rendering of a 4-panel visual dashboard with gridlines and legends.

Future Expansion Scope

The modular class structure allows seamless future extensions such as SQL database connectivity, interactive Streamlit/Dash web frontends, automated CSV export pipelines, and predictive machine learning models for revenue forecasting.

5. Hardware and Software requirements

Hardware Requirements

- Laptop or desktop computer.
- Minimum 4 GB RAM.
- Keyboard and mouse.
- Sufficient storage space.

Software Requirements

- Python programming language.
- Visual Studio Code (VS Code).
- Python interpreter.
- Python extension for VS Code.
- NumPy library.
- Pandas library.
- Matplotlib library.

6. Technologies and Libraries used

1. Python Programming Language

Serves as the core engine. Handles object instantiation, class inheritance hierarchy, control loops, and variable management.

2. Pandas Library (import pandas as pd)

Used for structured data tabularization. Converts raw dictionaries into 2D DataFrames, executes dropna() validation, and performs groupby() category aggregations.

3. NumPy Library (import numpy as np)

Provides high-performance vector calculations. Calculates statistical metrics including np.sum(), np.mean(), np.median(), and np.std().

4. Matplotlib Library (import matplotlib.pyplot as plt)

Handles dashboard rendering. Uses plt.subplots(2, 2) to draw line plots, bar charts, pie charts, and scatter plots within a unified canvas.

7. System Architecture & Object Design

The application follows strict Object-Oriented Principles to ensure modularity and separation of concerns. The data workflow is governed by a parent-child inheritance model.

8. Functional Work Algorithm

1. **Data Definition:** Initialize dictionary data containing 12 monthly transactional observations.
2. **Class Instantiation:** Instantiate RetailSalesAnalyzer(data), triggering the super().__init__(data) constructor in DatasetManager.
3. **DataFrame Conversion:** Transform dictionary structure into self.df DataFrame.
4. **Data Cleaning:** Call clean_data(), which executes self.df = self.df.dropna() to sanitize missing fields.
5. **Statistical Analysis:** Compute Total, Mean, Median, and Std Dev via compute_statistics() using NumPy.
6. **Dashboard Generation:** Group sales by category, set 2×2 subplots, render line/bar/pie/scatter figures, and call plt.show().

9. Python Source Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# --- 1. PARENT CLASS: Base Data Handler ---
class DatasetManager:
    def __init__(self, raw_data):
        print("--- Step 1: Initializing Retail Sales Dataset via Parent Class ---")
        self.raw_data = raw_data
        self.df = pd.DataFrame(self.raw_data)

    def clean_data(self):
        # Data Cleaning & Validation check (using re-assignment)
        self.df = self.df.dropna()
        return self.df

# --- 2. CHILD CLASS: Inherits from DatasetManager (Inheritance Concept) ---
class RetailSalesAnalyzer(DatasetManager):
    def __init__(self, raw_data):
        # Inherit initialization from Parent Class using super()
        super().__init__(raw_data)
        # Automatically clean data upon creation
        self.clean_data()

    def compute_statistics(self):
        print("\n--- Step 2: Statistical Computations using NumPy & Pandas ---")
```

```
    total_revenue = np.sum(self.df['Sales_Revenue'])
    mean_revenue = np.mean(self.df['Sales_Revenue'])
    median_revenue = np.median(self.df['Sales_Revenue'])
    std_revenue = np.std(self.df['Sales_Revenue'])

    print(f"Total Annual Revenue: ${total_revenue}")
    print(f"Mean Monthly Revenue: ${mean_revenue:.2f}")
    print(f"Median Monthly Revenue: ${median_revenue:.2f}")
    print(f"Standard Deviation of Revenue: ${std_revenue:.2f}")

def generate_visualizations(self):
    print("\n--- Step 3: Generating Visualizations using Matplotlib ---")
    # Setting up a 2x2 grid for subplots
    fig, axs = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle('Retail Sales & Customer Behavior EDA Dashboard', fontsize=16, fontweight='bold')

    # Simplified color list using easy-to-read names
    my_colors = ['lightcoral', 'skyblue', 'lightgreen']

    # 1. Line Plot: Sales Trends Over Time
    axs[0, 0].plot(self.df['Month'], self.df['Sales_Revenue'], marker='o', color='blue', linewidth=2)
    axs[0, 0].set_title('Sales Revenue Trend Over Time')
    axs[0, 0].set_xlabel('Month')
    axs[0, 0].set_ylabel('Revenue ($)')
    axs[0, 0].grid(True, linestyle='--', alpha=0.6)
    axs[0, 0].tick_params(axis='x', rotation=45)
```

```
        # 2. Bar Plot: Sales Performance Across Categories
        category_sales = self.df.groupby('Product_Category')['Sales_Revenue'].sum()
        category_sales.plot(kind='bar', ax=axs[0, 1], color=my_colors)
        axs[0, 1].set_title('Total Sales by Product Category')
        axs[0, 1].set_xlabel('Product Category')
        axs[0, 1].set_ylabel('Revenue ($)')
        axs[0, 1].grid(axis='y', linestyle='--', alpha=0.6)

        # 3. Pie Chart: Distribution of Sales by Category
        axs[1, 0].pie(category_sales, labels=category_sales.index, autopct='%1.1f%%', startangle=140,
                      colors=my_colors)
        axs[1, 0].set_title('Sales Distribution Percentage by Category')

        # 4. Scatter Plot: Customer Age vs Sales Revenue Correlation
        axs[1, 1].scatter(self.df['Customer_Age'], self.df['Sales_Revenue'], color='purple', s=100,
edgecolor='black')
        axs[1, 1].set_title('Customer Age vs Sales Revenue Correlation')
        axs[1, 1].set_xlabel('Customer Age')
        axs[1, 1].set_ylabel('Revenue ($)')
        axs[1, 1].grid(True, linestyle='--', alpha=0.6)

        # Adjust layout for neat presentation
        plt.tight_layout(pad=3.0, w_pad=4.0, h_pad=3.0, rect=[0, 0.03, 1, 0.93])

        print("\n--- Step 4: Displaying Graphs ---")
        plt.show()
```

```
# --- 3. MAIN EXECUTION ---
if __name__ == "__main__":
    # Your exact raw data dictionary kept completely as it is
    data = {
        'CustomerID': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112],
        'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'],
        'Product_Category': ['Electronics', 'Clothing', 'Home', 'Electronics', 'Clothing', 'Home',
                             'Electronics', 'Clothing', 'Home', 'Electronics', 'Clothing', 'Home'],
        'Sales_Revenue': [12000, 15000, 11000, 18000, 22000, 25000, 27000, 24000, 21000, 26000, 30000,
35000],
        'Units_Sold': [120, 150, 100, 180, 210, 240, 260, 230, 200, 250, 290, 340],
        'Customer_Age': [25, 30, 22, 35, 40, 28, 26, 32, 45, 29, 38, 33]
    }

    # Creating an object of the child class and calling its methods
    analyzer = RetailSalesAnalyzer(data)
    analyzer.compute_statistics()
    analyzer.generate_visualizations()
```

10. Output

```
PS C:\Users\muska\Pictures\muskan\uni notes\uni codes\python uni> &
uni codes/python uni/sales_project.py"
--- Step 1: Initializing Retail Sales Dataset via Parent Class ---

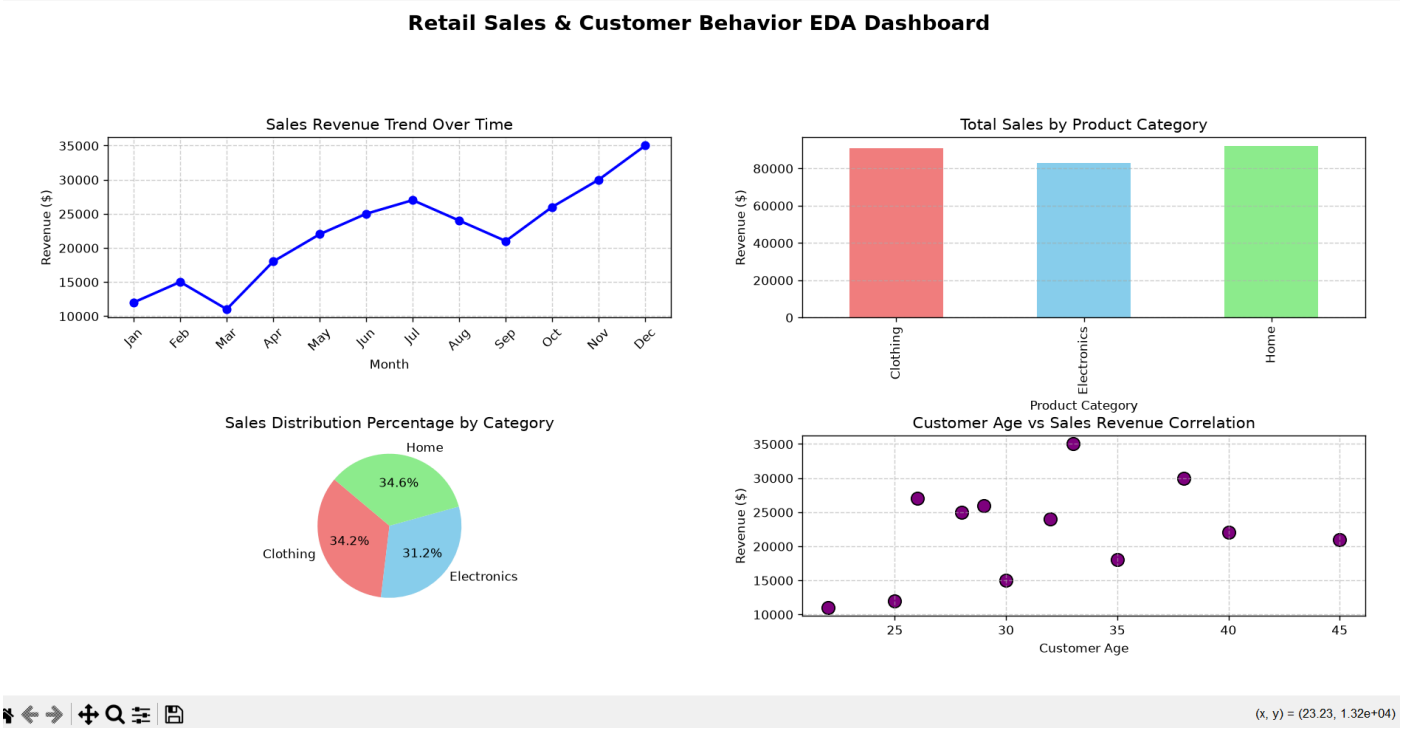
--- Step 2: Statistical Computations using NumPy & Pandas ---
Total Annual Revenue: $266000
Mean Monthly Revenue: $22166.67
Median Monthly Revenue: $23000.00
Standard Deviation of Revenue: $6914.16

--- Step 3: Generating Visualizations using Matplotlib ---

--- Step 4: Displaying Graphs ---
```

11. Graphical Dashboard Analysis

The 2×2 Matplotlib dashboard provides four distinct analytical views:



1. Sales Revenue Trend (Line Plot)

Tracks month-by-month financial growth. Demonstrates an upward trend peaking in Q4 (Dec revenue reaching $35,000), driven by holiday spending.

2. Category Performance (Bar Chart)

Aggregates absolute sales across categories. Compares Electronics, Clothing, and Home products to identify top-grossing inventory sectors.

3. Category Distribution (Pie Chart)

Illustrates percentage market share per product segment, providing immediate executive visibility into sector revenue dominance.

4. Age vs. Revenue (Scatter Plot)

Plots buyer age against transactional values. Analyzes demographic purchasing behavior to identify whether higher revenue correlates with specific age groups.

12. System Advantages

- **Automated Processing:** Eliminates manual calculation errors in annual total and average computations.
- **Modular Architecture:** Uses clear single-inheritance OOP design, allowing simple updates without disturbing the base data handler.
- **Multi-View Dashboard:** Consolidates line, bar, pie, and scatter visualizations into a single window layout.
- **Zero External Overhead:** Relies entirely on core Python libraries (Pandas, NumPy, Matplotlib) without requiring complex database installation.

13. Project Limitations

- **Static Dictionary Input:** Data is directly initialized inside code rather than reading dynamically from external CSV/SQL sources.
- **No Desktop GUI Controls:** Relies on terminal execution and Matplotlib window controls rather than interactive Tkinter buttons.
- **Single-Year Scope:** The current dataset focuses on 12 months without multi-year seasonal comparisons.

14. Key Learning Outcomes

Through the development of this project, the following core MCA Data Science capabilities were gained:

- Designing class hierarchies and invoking parent methods using super().__init__().
- Cleaning Pandas DataFrames safely through non-destructive re-assignment (df = df.dropna()).
- Computing mathematical dispersion statistics using vector-accelerated NumPy commands.
- Configuring complex 2×2 subplot canvases with custom color schemes, legends, gridlines, and rotated labels.

15. Conclusion

The **Retail Sales & Customer Behavior Exploratory Data Analysis (EDA) System** successfully demonstrates how fundamental Python programming techniques can be seamlessly blended with data science libraries to solve practical business analytics problems.

By structuring the application around Object-Oriented Principles—specifically utilizing the DatasetManager parent class and RetailSalesAnalyzer child class—the project maintains clean separation between raw data handling, data validation, mathematical calculations, and dashboard visualization.

The automated computing pipeline replaces tedious manual calculations with exact NumPy statistical calculations ($266,000 total revenue, $22,166.67 monthly mean), while the 4-panel Matplotlib dashboard provides instant visual insights into Q4 revenue surges and category distribution.